

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC KINH TẾ TP HỒ CHÍ MINH (UEH)
TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ

ĐỒ ÁN MÔN HỌC

ĐỀ TÀI

**Xây dựng Chương trình quản lý logistic (cho các công ty
chuyển hàng)**

Học Phần: Lập Trình Hướng Đối Tượng

Danh Sách Nhóm:

- 1 NGUYỄN HOÀNG ANH
- 2 LỮ VÕ HOÀNG PHÚC
- 3 TỪ TRƯỞNG THANH TRÚC

Chuyên Ngành: KỸ THUẬT PHẦN MỀM

Khóa: K50

Giảng Viên: TS. Đặng Ngọc Hoàng Thành

Tp. Hồ Chí Minh, Ngày xx tháng 12 năm 2025

Mục lục

1	PHẦN MỞ ĐẦU	1
1.1	Tính cấp thiết của đề tài	1
1.1.1	Vấn đề thực tiễn hiện nay	1
1.1.2	Lợi ích khi áp dụng phần mềm quản lý	2
1.1.3	Mục tiêu cụ thể của đề tài	2
1.2	Giới thiệu về Kỹ thuật lập trình hướng đối tượng (LTHĐT)	3
1.2.1	Định nghĩa	3
1.2.2	Nguyên tắc cơ bản của OOP	4
1.2.3	Lợi ích khi dùng OOP	4
1.3	Tầm quan trọng của LTHĐT trong Kỹ thuật phần mềm	5
2	PHÂN TÍCH VÀ THIẾT KẾ LỚP	6
2.1	Phân tích bài toán	6
2.2	Thiết kế lớp và Sơ đồ lớp (Class diagrams)	7
2.2.1	Thiết kế lớp	7
2.2.2	Sơ đồ lớp	8
2.2.3	Các tính chất cơ bản của Lập trình hướng đối tượng (OOP)	15
2.2.4	Design Pattern	20
2.2.5	Các quan hệ trong Lập trình hướng đối tượng	23
2.2.6	Kỹ thuật Serialization và Deserialization	25
3	XÂY DỰNG ỨNG DỤNG	27
3.1	Thiết kế giao diện chương trình	27
3.1.1	Giao diện đăng nhập, đăng ký	27
3.1.2	Giao diện tổng quan	27
3.1.3	Giao diện quản lý đơn hàng	27
3.1.4	Giao diện quản lý khách hàng	27
3.1.5	Giao diện quản lý nhân sự	27
3.1.6	Giao diện quản lý phương tiện	27
3.1.7	Giao diện quản lý kho bãi	27
3.1.8	Giao diện điều phối giao hàng	27
3.1.9	Giao diện quản lý chứng từ	27
3.1.10	Giao diện báo cáo, thống kê	27
3.2	Phát triển các chức năng của ứng dụng	27
3.2.1	Quản lý tài khoản và phân quyền	27
3.2.2	Quản lý khách hàng	27
3.2.3	Quản lý nhân sự	27
3.2.4	Quản lý phương tiện	27
3.2.5	Quản lý kho bãi	27

3.2.6	Quản lý đơn hàng và kiện hàng	27
3.2.7	Điều phối tài xế và phương tiện	27
3.2.8	Quản lý thanh toán và hóa đơn	27
3.3	Các kịch bản thực thi ứng dụng	27
4	THẢO LUẬN & ĐÁNH GIÁ	28
4.1	Các kết quả nhận được	28
4.2	Một số tồn tại	28
4.3	Hướng phát triển	28
	Tài liệu tham khảo	29
	Phụ lục	30
	Phụ lục hình ảnh	31
	Bảng phân công nhiệm vụ	32
	Link đồ án	32

1 PHẦN MỞ ĐẦU

1.1 Tính cấp thiết của đề tài

Sự phát triển bùng nổ của thương mại toàn cầu và nền kinh tế số đang đặt ngành logistics trước những cơ hội và thách thức chưa từng có. Trong bối cảnh đó, việc nghiên cứu và “Xây dựng chương trình quản lý logistics” cho các công ty chuyển hàng không còn là một lựa chọn để nâng cấp tiện ích, mà đã trở thành yêu cầu sinh tồn. Tính cấp thiết của đề tài này được thể hiện rõ nét qua lăng kính từ bối cảnh toàn cầu đến thực tiễn thị trường tại Việt Nam:

1.1.1 Vấn đề thực tiễn hiện nay

1. Bối cảnh Logistics Toàn cầu: Sức ép từ sự bất định và công nghệ số

- **Sự đứt gãy và tái cấu trúc chuỗi cung ứng:** Những biến động địa chính trị liên tục và xu hướng đa dạng hóa nguồn cung khiến chuỗi cung ứng trở nên phức tạp. Các công ty chuyển hàng cần một hệ thống có khả năng dự báo, lập kế hoạch linh hoạt và thay đổi lộ trình theo thời gian thực.
- **Chuyển đổi số và tự động hóa:** Các tập đoàn logistics quốc tế đang ứng dụng mạnh mẽ Trí tuệ nhân tạo (AI), Internet vạn vật (IoT) và Dữ liệu lớn (Big Data) vào vận hành. Việc thiếu vắng một chương trình quản lý lõi sẽ khiến các doanh nghiệp vận tải tụt hậu hoàn toàn trong cuộc đua về tốc độ xử lý đơn hàng và tối ưu hóa không gian lưu kho.
- **Tiêu chuẩn Logistics Xanh (Green Logistics):** Áp lực giảm phát thải carbon từ các hiệp định thương mại quốc tế buộc các công ty phải tối ưu hóa quãng đường vận chuyển, giảm thiểu tình trạng xe hoặc container chạy rỗng quay đầu. Lượng khí thải thường được đánh giá qua các mô hình toán học cơ bản

$$E = \sum_{i=1}^n (d_i \times f_i)$$

đóng vai trò đặc biệt quan trọng để đánh giá hiệu suất. Chỉ có hệ thống phần mềm tính toán thông minh mới có thể gộp chuyển và phân luồng hiệu quả.

2. Bối cảnh Logistics tại Việt Nam: Tiềm năng bùng nổ nhưng chi phí bào mòn

- **Động lực từ Thương mại điện tử (TMĐT):** Giai đoạn hiện tại, quy mô thị trường TMĐT Việt Nam tiếp tục bùng nổ. Sự thống trị của các nền tảng mua sắm trực tuyến đòi hỏi tốc độ giao hàng chặng cuối (last-mile delivery) cực nhanh và khả năng xử lý hàng triệu đơn hàng nhỏ lẻ mỗi ngày.

- **Ngịch lý “Quy mô lớn - Chi phí cao”:** Mặc dù thị trường logistics Việt Nam có quy mô lớn, nhưng chi phí logistics vẫn chiếm tỷ trọng rất cao, xấp xỉ

$$C = \frac{16}{100} - \frac{18}{100} \text{ GDP}$$

lớn hơn nhiều so với mức trung bình của các nước phát triển. Nguyên nhân cốt lõi là sự manh mún, thiếu kết nối thông tin đồng bộ giữa chủ hàng, nhà xe và kho bãi.

- **Sự yếu thế của doanh nghiệp nội địa:** Hầu hết các công ty chuyển hàng tại Việt Nam là doanh nghiệp vừa và nhỏ, thường phải làm thầu phụ cho các tập đoàn nước ngoài. Việc tự xây dựng hoặc áp dụng các chương trình quản lý logistics chuyên sâu là bước đi bắt buộc để doanh nghiệp nội nâng cao năng lực cạnh tranh.

1.1.2 Lợi ích khi áp dụng phần mềm quản lý

Từ hai bối cảnh trên, việc xây dựng một chương trình quản lý logistics sẽ mang lại các giá trị định lượng và định tính rõ ràng:

- **Tối ưu hóa chi phí vận hành:** Phần mềm tự động hóa việc lên lịch trình, sắp xếp tải trọng xe, quản lý luân chuyển hàng hóa tại kho, từ đó cắt giảm trực tiếp các chi phí lãng phí như nhiên liệu, nhân công và thời gian chờ.
- **Nâng cao trải nghiệm khách hàng:** Cung cấp nền tảng để khách hàng tra cứu trạng thái đơn hàng mọi lúc, mọi nơi, tăng tính minh bạch và độ tin cậy.
- **Hỗ trợ ra quyết định bằng Dữ liệu (Data-Driven):** Thay vì quản lý bằng giấy tờ rời rạc dễ sai sót, hệ thống cung cấp các báo cáo trực quan về doanh thu, năng suất, giúp Ban lãnh đạo phản ứng nhanh với thị trường.
- **Sẵn sàng cho các hành lang pháp lý mới:** Khi nhà nước áp dụng các khung pháp lý chặt chẽ hơn về kiểm soát hàng hóa và hóa đơn điện tử, một hệ thống chuẩn hóa dữ liệu là bắt buộc để duy trì vận hành thông suốt.

1.1.3 Mục tiêu cụ thể của đề tài

Tóm lại, đề tài "Xây dựng Chương trình quản lý logistics cho các công ty chuyển hàng" hướng đến các mục tiêu cụ thể sau:

- **Phân tích và hệ thống hóa** các yêu cầu nghiệp vụ thực tế của doanh nghiệp chuyển hàng vừa và nhỏ tại Việt Nam trong bối cảnh chuyển đổi số.
- **Thiết kế và xây dựng** chương trình quản lý logistics tích hợp, bao gồm các phân hệ: quản lý đơn hàng, lịch trình vận chuyển, kho bãi và báo cáo doanh thu.

- **Tối ưu hóa chi phí vận hành** thông qua các thuật toán lên lịch trình và gộp chuyển thông minh, góp phần giảm thiểu chi phí logistics xuống mức cạnh tranh hơn so với mặt bằng chung.
- **Nâng cao năng lực cạnh tranh** cho doanh nghiệp nội địa, không chỉ mang tính ứng dụng thực tiễn tức thời mà còn mang tính thời sự sâu sắc, đáp ứng chính xác nhu cầu chuyển đổi số tất yếu của dòng chảy thương mại hiện đại.

1.2 Giới thiệu về Kỹ thuật lập trình hướng đối tượng (LTHĐT)

1.2.1 Định nghĩa

Kỹ thuật Lập trình hướng đối tượng (Object-Oriented Programming - OOP) là một phương pháp luận lập trình tiên tiến, nền tảng của kỹ thuật phần mềm hiện đại. Thay vì tổ chức chương trình xoay quanh các chuỗi hành động hay logic xử lý tuần tự như lập trình thủ tục (Procedural Programming), OOP tập trung vào việc mô hình hóa hệ thống thông qua các “đối tượng” (objects). Mỗi đối tượng là một thực thể hoàn chỉnh, bao bọc cả trạng thái (dữ liệu/thuộc tính) và hành vi (phương thức/hàm). Phương pháp này cho phép lập trình viên ánh xạ trực tiếp các thực thể từ thế giới thực vào trong môi trường tính toán một cách logic và trực quan.

Lịch sử hình thành và quá trình phát triển

Kỹ thuật OOP không xuất hiện một cách ngẫu nhiên mà là thành tựu của quá trình tiến hóa dài hạn trong khoa học máy tính, nhằm giải quyết “cuộc khủng hoảng phần mềm” (software crisis) khi quy mô của các chương trình ngày càng trở nên khổng lồ và phức tạp:

- **Thập niên 1960 - Khởi nguồn ý tưởng:** Khái niệm về đối tượng lần đầu tiên được giới thiệu trong ngôn ngữ Simula 67 (phát triển bởi Ole-Johan Dahl và Kristen Nygaard tại Na Uy). Ngôn ngữ này ban đầu được thiết kế để phục vụ các hệ thống mô phỏng chuyên dụng, nơi khái niệm về “lớp” (class) và “đối tượng” (object) bắt đầu được định hình rõ rệt.
- **Thập niên 1970 - Định hình nền tảng:** Ngôn ngữ Smalltalk, được phát triển tại trung tâm nghiên cứu Xerox PARC bởi Alan Kay và các cộng sự, đã đưa OOP trở thành một mô hình lập trình hoàn chỉnh và độc lập. Alan Kay cũng chính là người chính thức đặt ra thuật ngữ “Object-Oriented Programming”.
- **Thập niên 1980 - Bước ngoặt thương mại hóa:** Bjarne Stroustrup đã sáng tạo ra C++ bằng cách tích hợp các đặc trưng hướng đối tượng vào ngôn ngữ C truyền thống. Sự giao thoa này giúp OOP nhanh chóng được chấp nhận và ứng dụng rộng rãi trong giới công nghiệp phần mềm toàn cầu.
- **Từ thập niên 1990 đến nay - Kỷ nguyên bùng nổ:** Sự ra đời của Java, C#, và sau này là các ngôn ngữ linh hoạt như Python, Ruby đã biến OOP trở thành tiêu chuẩn công nghiệp (industry standard). Ngày nay, hầu hết các hệ thống phần mềm doanh nghiệp quy mô lớn đều được xây dựng trên nền tảng kiến trúc này.

1.2.2 Nguyên tắc cơ bản của OOP

Sức mạnh và khả năng tái sử dụng mã nguồn của kỹ thuật lập trình hướng đối tượng được thiết lập trên bốn trụ cột cơ sở:

- **Tính đóng gói (Encapsulation):** Cơ chế che giấu trạng thái nội tại của đối tượng, chỉ cho phép giao tiếp thông qua các giao diện (interface) hoặc phương thức công khai (public methods) được định nghĩa sẵn. Điều này giúp bảo vệ tính toàn vẹn của hệ thống dữ liệu khỏi những can thiệp không hợp lệ từ bên ngoài.
- **Tính kế thừa (Inheritance):** Cho phép một lớp đối tượng mới (derived class) kế thừa nguyên bản các thuộc tính và phương thức của một lớp đã tồn tại (base class). Cơ chế này không chỉ giúp tối ưu hóa việc tái sử dụng mã nguồn mà còn thiết lập các mối quan hệ phân cấp tự nhiên.
- **Tính đa hình (Polymorphism):** Đặc tính cho phép các đối tượng thuộc các lớp khác nhau phản hồi cùng một thông điệp (lời gọi hàm) theo những phương thức đặc thù riêng biệt, làm tăng tính linh hoạt và khả năng mở rộng của chương trình.
- **Tính trừu tượng (Abstraction):** Kỹ thuật cô lập và chỉ tập trung vào những đặc điểm thiết yếu nhất của một đối tượng, đồng thời loại bỏ các chi tiết cài đặt phức tạp bên dưới, giúp giảm tải rủi ro về mặt logic khi thiết kế kiến trúc hệ thống.

1.2.3 Lợi ích khi dùng OOP

Việc áp dụng phương pháp luận OOP vào đề tài “Xây dựng Chương trình quản lý logistics” là một lựa chọn tối ưu về mặt kiến trúc. Các thực thể cốt lõi trong mạng lưới chuỗi cung ứng như *Đơn hàng (Order)*, *Xe tải vận chuyển (Truck)*, *Khách hàng (Customer)*, hay *Kho bãi (Warehouse)* có thể được ánh xạ trực tiếp thành các Lớp (Classes) trong phần mềm, mang lại những lợi ích thiết thực sau:

- **Tái sử dụng và bảo trì mã nguồn dễ dàng:** Nhờ cơ chế kế thừa, các lớp dùng chung có thể được kế thừa và mở rộng mà không cần viết lại từ đầu, giúp tiết kiệm đáng kể thời gian phát triển và chi phí bảo trì về lâu dài.
- **Tính mô-đun hóa cao:** Mỗi lớp đối tượng hoạt động như một mô-đun độc lập, cho phép đội ngũ phát triển làm việc song song trên nhiều phần khác nhau của hệ thống mà không gây xung đột.
- **Linh hoạt trong mở rộng chức năng:** Việc bảo trì, nâng cấp hệ thống (chẳng hạn như thêm các loại phương tiện vận tải mới hoặc bổ sung thuật toán xếp đồ) có thể được thực hiện độc lập nhờ tính đóng gói và kế thừa, mà không làm xáo trộn hay phá vỡ kiến trúc tổng thể của chương trình.

- **Phản ánh trực quan thực thể nghiệp vụ:** Cấu trúc hướng đối tượng cho phép lập trình viên mô hình hóa trực tiếp các quy trình nghiệp vụ logistics vào trong mã nguồn, giúp hệ thống dễ hiểu, dễ kiểm thử và dễ bàn giao hơn cho các bên liên quan.

1.3 Tầm quan trọng của LTHĐT trong Kỹ thuật phần mềm

Trong lĩnh vực Kỹ thuật phần mềm, LTHĐT không chỉ là một kỹ thuật viết mã đơn thuần mà đã trở thành một hệ tư tưởng thiết kế mang tính nền tảng. Khi quy mô phần mềm ngày càng mở rộng và yêu cầu nghiệp vụ trở nên phức tạp, việc áp dụng LTHĐT đóng vai trò then chốt trong việc đảm bảo chất lượng và khả năng vận hành lâu dài của hệ thống, thể hiện qua các khía cạnh sau:

- **Quản trị độ phức tạp của hệ thống (Managing Complexity):** Thay vì phải đối mặt với một khối logic khổng lồ và đan xen chằng chịt, LTHĐT cho phép các kỹ sư phần mềm phân rã một hệ thống lớn thành các thực thể (đối tượng) nhỏ hơn, độc lập và dễ kiểm soát. Mỗi đối tượng tự quản lý trạng thái và hành vi của riêng nó, giúp thu hẹp phạm vi lỗi và dễ dàng khoanh vùng khi xảy ra sự cố trong quá trình kiểm thử (debugging).
- **Tối ưu hóa khả năng tái sử dụng mã nguồn (Code Reusability):** Thông qua cơ chế kế thừa (Inheritance) và các mẫu thiết kế (Design Patterns), LTHĐT cho phép lập trình viên tái sử dụng các mô-đun mã đã được xây dựng, kiểm thử và nghiệm thu. Điều này giúp giảm thiểu việc viết lại các đoạn mã trùng lặp, từ đó rút ngắn đáng kể vòng đời phát triển phần mềm (Software Development Life Cycle - SDLC) và tiết kiệm chi phí nhân sự.
- **Nâng cao khả năng bảo trì và linh hoạt mở rộng (Maintainability and Scalability):** Nhờ tính đóng gói (Encapsulation), những thay đổi trong cấu trúc nội bộ hoặc thuật toán của một lớp (class) sẽ không gây ra hiệu ứng dây chuyền làm sụp đổ các thành phần khác của hệ thống, miễn là giao diện (interface) giao tiếp được giữ nguyên. Khi cần tích hợp thêm tính năng mới, nhà phát triển chỉ cần mở rộng các lớp hiện có mà không làm phá vỡ kiến trúc nền tảng ban đầu.
- **Mô hình hóa thế giới thực trực quan (Real-world Modeling):** LTHĐT thu hẹp khoảng cách giữa bài toán nghiệp vụ trong thực tế và các dòng mã lập trình. Việc ánh xạ các khái niệm thực tế thành các đối tượng phần mềm giúp các nhà phân tích hệ thống, lập trình viên và chuyên gia nghiệp vụ (domain experts) dễ dàng tìm được tiếng nói chung trong quá trình đặc tả yêu cầu và thiết kế.
- **Hỗ trợ làm việc nhóm hiệu quả (Team Collaboration):** Kiến trúc hướng đối tượng tạo ra các ranh giới chức năng rõ ràng giữa các phân hệ. Nhờ đó, một đội ngũ kỹ sư có thể phân chia công việc, phát triển song song nhiều module độc lập mà không gặp phải hiện tượng xung đột mã nguồn nghiêm trọng khi tiến hành tích hợp hệ thống (System Integration).

Đối chiếu với đề tài quản lý logistics, việc ứng dụng triết lý LTHĐT là giải pháp kỹ thuật tối ưu để kiến tạo một chương trình đạt tiêu chuẩn công nghiệp: bền vững, an toàn dữ liệu và sẵn sàng mở rộng quy mô trong tương lai.

2 PHÂN TÍCH VÀ THIẾT KẾ LỚP

2.1 Phân tích bài toán

Trong bối cảnh thương mại điện tử và dịch vụ giao nhận phát triển mạnh, bài toán quản lý logistics không chỉ dừng lại ở việc lưu thông tin đơn hàng mà còn phải kiểm soát toàn bộ vòng đời vận chuyển: tiếp nhận đơn, quản lý kiện hàng, điều phối tài xế, phân bổ phương tiện, nhập xuất kho, theo dõi trạng thái giao hàng, xử lý thanh toán COD, ghi nhận sự cố và lập báo cáo vận hành. Nếu các chức năng này được cài đặt rời rạc, hệ thống sẽ khó mở rộng, khó bảo trì và dễ phát sinh sai lệch dữ liệu giữa đơn hàng, kho, tài xế và phương tiện.

Dự án **Logistics System** được xây dựng theo hướng lập trình hướng đối tượng (OOP), trong đó các thực thể nghiệp vụ được mô hình hóa thành lớp, mỗi lớp giữ trạng thái và hành vi riêng. Hệ thống tách rõ phần lõi nghiệp vụ `Logistics.Core` và phần giao diện `Logistics.WinFormsUI`; dữ liệu được lưu dưới dạng JSON thông qua Repository, còn giao diện WinForms chỉ gọi Service và DTO để thao tác với người dùng.

Các nhu cầu chính mà hệ thống cần đáp ứng gồm:

- Quản lý tài khoản, phân quyền và phiên đăng nhập cho Admin, Dispatcher, Driver, Warehouse Staff và Customer.
- Quản lý khách hàng, nhân viên, tài xế, điều phối viên và nhân viên kho.
- Tạo, cập nhật, hủy và theo dõi trạng thái đơn hàng theo mã vận đơn.
- Quản lý danh sách kiện hàng, khối lượng thực tế, khối lượng quy đổi, hàng dễ vỡ, giá trị hàng và vị trí trong kho.
- Tính phí vận chuyển theo loại dịch vụ Standard, Express hoặc Instant.
- Điều phối tài xế và phương tiện dựa trên trạng thái sẵn sàng, tải trọng và thể tích còn cho phép.
- Quản lý kho, sức chứa, nhập xuất kiện hàng và nhật ký tồn kho.
- Quản lý phương tiện, động cơ, lịch sử bảo trì và trạng thái vận hành.
- Ghi nhận giao dịch, báo cáo sự cố, lịch sử trạng thái và sổ liệu dashboard.
- Lưu trữ, khôi phục và bảo toàn dữ liệu bằng kỹ thuật serialization/deserialization trên JSON.

Từ các chức năng trên, hệ thống được chia thành các nhóm lớp chính:

- **Nhóm tác nhân:** `Person`, `Customer`, `Staff`, `Admin`, `Driver`, `Dispatcher`, `WarehouseStaff`.

- **Nhóm nghiệp vụ vận chuyển:** Order, Package, DeliveryRoute, OrderDetail, OrderStatusHistory, DeliveryTrip, ShipmentLog, Transaction, ProblemReport, AuditLog.
- **Nhóm hạ tầng:** Vehicle, Engine, Warehouse, WarehouseLocation, Equipment, MaintenanceLog, VehicleAssignment, WarehouseInventoryLog.
- **Nhóm tài khoản và bảo mật:** User, LoginCredentials, UserRole, SessionManager, RoleGuard.
- **Nhóm truy cập dữ liệu và xử lý nghiệp vụ:** IRepository<T>, JsonRepository<T>, các repository cụ thể, các service interface và service implementation.

2.2 Thiết kế lớp và Sơ đồ lớp (Class diagrams)

2.2.1 Thiết kế lớp

Thiết kế lớp của hệ thống được xây dựng theo nguyên tắc mỗi lớp chỉ chịu trách nhiệm cho một nhóm thông tin và hành vi rõ ràng. Các lớp model trong `Logistics.Core.Models` giữ dữ liệu và các hành vi nghiệp vụ trực tiếp của đối tượng. Các lớp service trong `Logistics.Core.Services` điều phối luồng xử lý phức tạp giữa nhiều model và repository. Các lớp repository đảm nhiệm đọc/ghi dữ liệu, tránh để giao diện hoặc service thao tác trực tiếp với file JSON.

a. Nhóm lớp tác nhân

Person là lớp trừu tượng mô tả thông tin chung của con người như mã định danh, họ tên, số điện thoại, email, ngày sinh, giới tính và địa chỉ. Từ lớp này, hệ thống tách ra Customer cho khách hàng và Staff cho nhân sự nội bộ. Staff tiếp tục là lớp trừu tượng cho các vai trò chuyên biệt gồm Admin, Driver, Dispatcher và WarehouseStaff.

Cách tổ chức này giúp tái sử dụng các thuộc tính chung ở lớp cha, đồng thời cho phép từng lớp con mở rộng dữ liệu riêng. Ví dụ Driver có bằng lái, trạng thái tài xế, vị trí hiện tại, số chuyến giao hàng và danh sách xe được phân công; WarehouseStaff có mã kho, ca làm việc và phụ cấp ca; Dispatcher có khu vực quản lý và thưởng KPI.

b. Nhóm lớp đơn hàng và kiện hàng

Order là lớp trung tâm của nghiệp vụ vận chuyển. Mỗi đơn hàng có mã vận đơn, người gửi, người nhận, tuyến giao hàng, danh sách kiện hàng, danh sách chi tiết đơn, lịch sử trạng thái, tổng khối lượng, tổng chi phí, loại dịch vụ và thông tin tài xế/phương tiện được giao. Lớp này hiện thực ITrackable, IReportable và ISerializable, thể hiện rằng đơn hàng có thể theo dõi, sinh báo cáo và lưu/khôi phục dữ liệu.

Package mô tả từng kiện hàng trong đơn, bao gồm mã kiện, mã đơn, mô tả, khối lượng thực tế, kích thước, khối lượng quy đổi, trạng thái, kho hiện tại và vị trí kệ. Lớp này có các hành

vi như tính khối lượng quy đổi, lấy khối lượng tính cước, đánh dấu đã lấy hàng, nhập kho, xuất kho, đang giao, đã giao, hoàn trả, thất lạc hoặc hư hỏng.

DeliveryRoute, OrderDetail, OrderStatusHistory, DeliveryTrip, ShipmentLog, Transaction, ProblemReport và AuditLog bổ sung các phần nghiệp vụ còn lại: tuyến giao hàng, chi tiết đơn, lịch sử trạng thái, chuyển giao, nhật ký theo dõi, giao dịch thanh toán, báo cáo sự cố và nhật ký kiểm toán.

c. Nhóm lớp hạ tầng logistics

Vehicle quản lý phương tiện vận chuyển với loại xe, tải trọng, thể tích, kích thước, số km, nhiên liệu, trạng thái, danh sách tài xế được phép vận hành, lịch sử bảo trì và động cơ. Lớp này có phương thức kiểm tra khả năng chở hàng thông qua CanCarry(weight, volume), cập nhật nhiên liệu, cập nhật số km, đưa xe vào bảo trì và hoàn tất bảo trì.

Warehouse quản lý kho hàng với mã kho, tên, địa chỉ, tọa độ, loại kho, tổng sức chứa, sức chứa đã dùng, giờ hoạt động và quản lý kho. Các lớp WarehouseLocation, WarehouseInventoryLog và Equipment mô tả vị trí lưu trữ, nhật ký nhập xuất kho và thiết bị trong kho. Nhóm này giúp hệ thống kiểm soát việc nhập/xuất kiện hàng và năng lực lưu trữ.

d. Nhóm lớp tài khoản, bảo mật và phiên đăng nhập

User lưu thông tin đăng nhập, mật khẩu băm, salt, vai trò, câu hỏi bảo mật, người liên kết, ngày tạo và trạng thái tài khoản. LoginCredentials là đối tượng truyền dữ liệu đăng nhập từ giao diện vào tầng xác thực, còn UserRole định nghĩa các vai trò tài khoản. SessionManager giữ người dùng hiện tại trong toàn ứng dụng, còn RoleGuard gom các hàm kiểm tra quyền như quyền điều phối, quản lý kho, quản lý xe, quản lý nhân viên và xem báo cáo.

e. Nhóm lớp dịch vụ, repository, DTO và mapping

Tầng repository sử dụng interface IRepository<T> và lớp generic JsonRepository<T> để chuẩn hóa thao tác lấy tất cả, lấy theo ID, thêm, cập nhật, xóa, lưu và nạp dữ liệu. Các repository cụ thể như OrderRepository, VehicleRepository, WarehouseRepository, CustomerRepository, DriverRepository kế thừa hoặc sử dụng hành vi chung để xử lý từng loại dữ liệu.

Tầng service hiện có 17 interface và 18 lớp triển khai, gồm OrderService, DispatchService, WarehouseService, VehicleService, AuthService, ReportService, TransactionService và các service khác. DTO và mapping extension được dùng để chuyển model nghiệp vụ sang dữ liệu hiển thị, giúp giao diện không phụ thuộc trực tiếp vào toàn bộ model nội bộ.

2.2.2 Sơ đồ lớp

Sơ đồ lớp của hệ thống tập trung vào các lớp domain chính trong Logistics.Core.Models. Các lớp WinForms như form, user control, helper giao diện không đưa vào sơ đồ tổng quát để tránh làm nhiễu thiết kế nghiệp vụ.

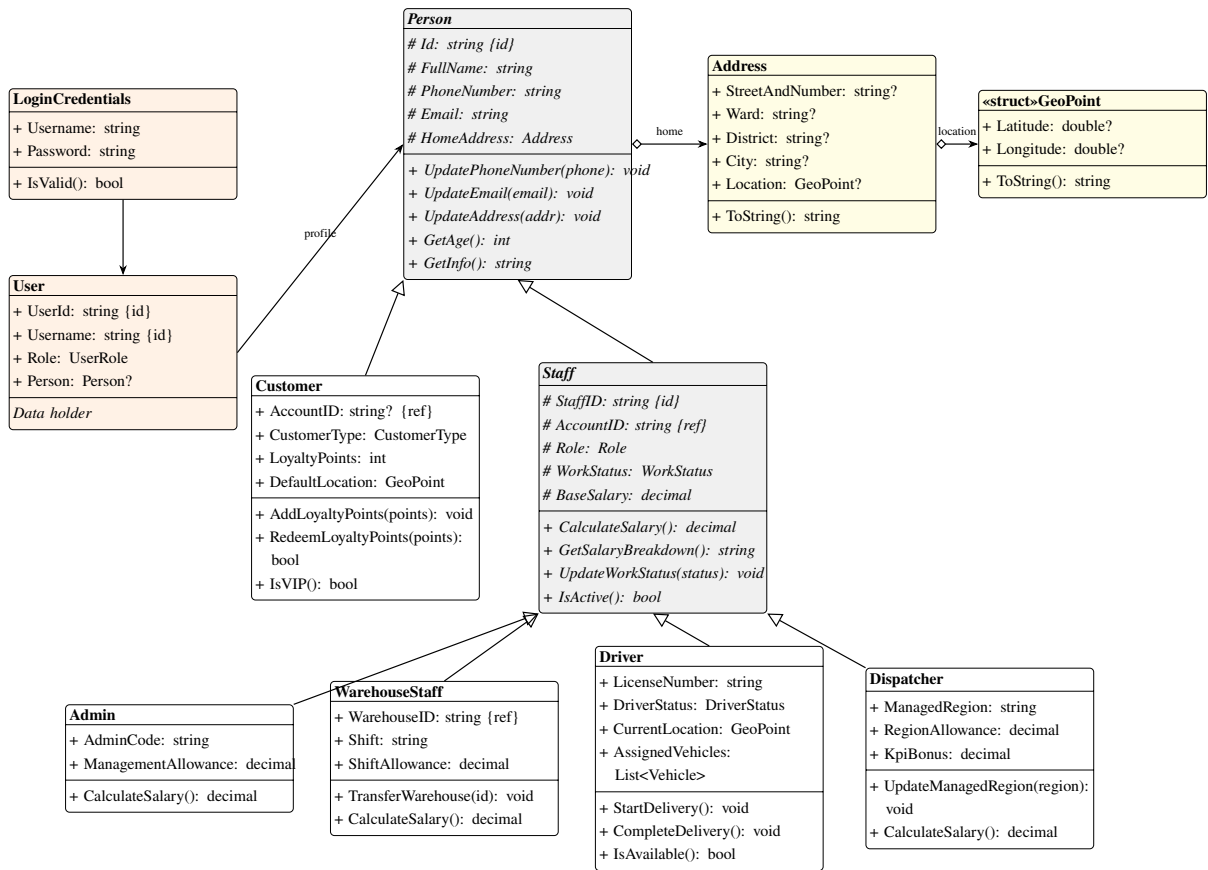
Quy ước trình bày UML. Sơ đồ bên dưới dùng ký hiệu + cho thành phần public, # cho protected, - cho private/internal implementation detail. Vì hệ thống lưu bằng JSON chứ không dùng cơ sở dữ liệu quan hệ, các trường như `TrackingNumber`, `VehicleID`, `WarehouseID` không được gọi là khóa chính theo nghĩa SQL. Báo cáo ký hiệu {id} cho trường định danh nghiệp vụ và {ref} cho trường tham chiếu tới định danh của lớp khác. Đây là cách diễn đạt tương đương khóa chính/khóa ngoại ở mức domain model nhưng vẫn đúng bản chất lập trình hướng đối tượng. Để sơ đồ dễ đọc, các quan hệ phụ thuộc bằng nét đứt được giải thích bằng bảng {ref} thay vì vẽ chồng lên sơ đồ.

Nhóm thành phần	Số lượng	Ghi chú
File C# nguồn	143	Tập tin nguồn của dự án trong <code>Logistics.Core</code> .
Các Lớp (Classes)	121	Bao gồm domain models, DTOs, các services, repositories, validators, mappings, v.v.
Giao diện (Interfaces)	22	Các interface phục vụ trừu tượng hóa nghiệp vụ và truy xuất dữ liệu.
Kiểu Liệt kê (Enums)	22	Định nghĩa các trạng thái, phân quyền tài khoản, loại xe, loại kho, hình thức COD.
Delegate	1	<code>OrderStatusChangedEventHandler</code> đóng vai trò làm delegate cho Observer Pattern.
Domain Models	29	Các lớp thực thể cốt lõi thuộc phân hệ nghiệp vụ vận chuyển và hạ tầng.
Model Interfaces	3	Các giao diện nghiệp vụ dùng chung: <code>ITrackable</code> , <code>IReportable</code> , <code>ISalaryCalculable</code> .

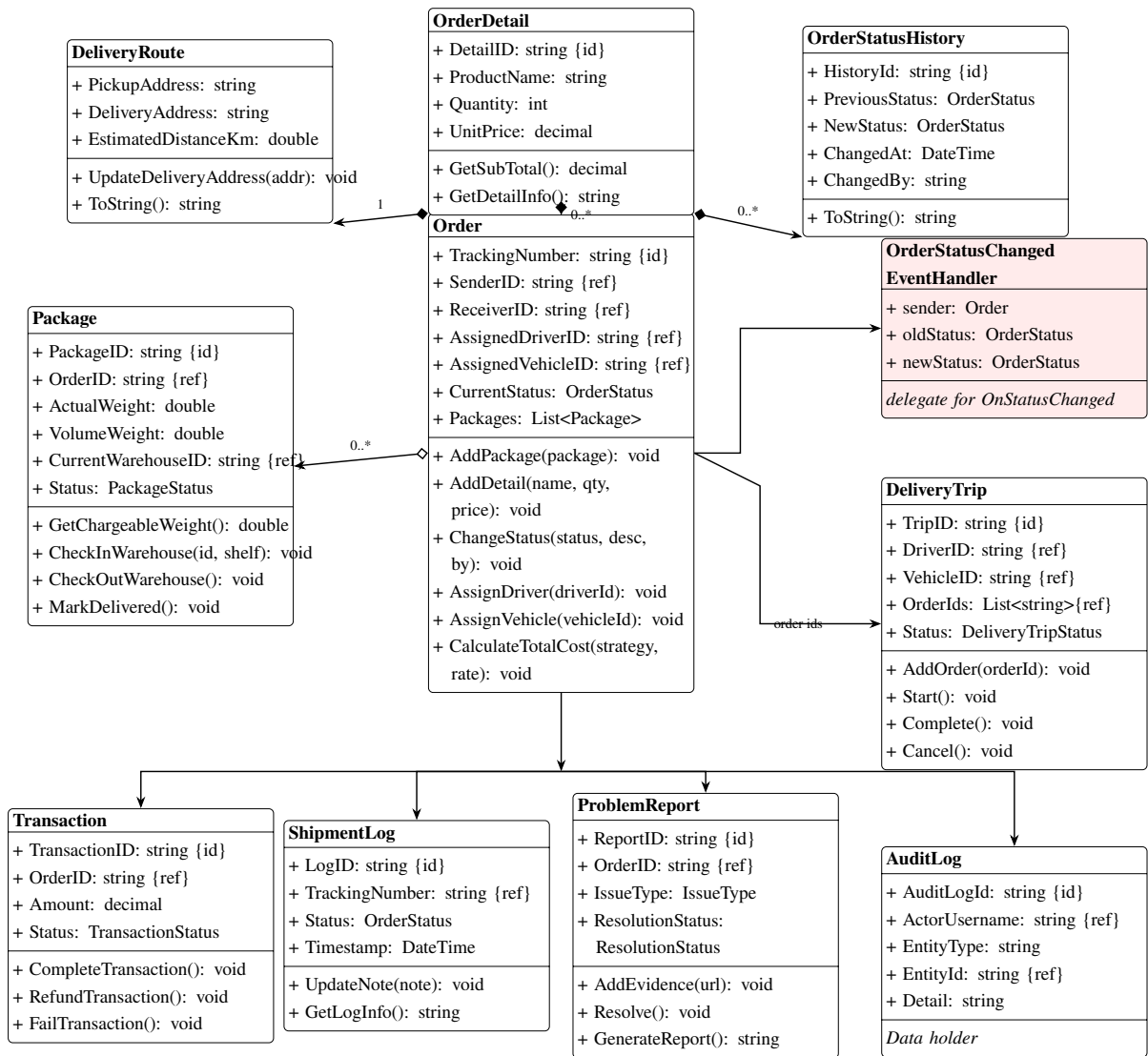
Bảng 1: Bảng thống kê thành phần cấu trúc của hệ thống Logistics

Lớp	Trường định danh {id}	Trường tham chiếu {ref}
Person	Id	HomeAddress tham chiếu đối tượng Address.
User	UserId, Username	Person liên kết hồ sơ tác nhân.
Staff	StaffID	AccountID liên kết tài khoản.
Order	TrackingNumber	SenderID, ReceiverID, AssignedDriverID, AssignedVehicleID.
Package	PackageID	OrderID, CurrentWarehouseID.
DeliveryTrip	TripID	DriverID, VehicleID, OrderIds.
Transaction	TransactionID	OrderID.
ProblemReport	ReportID	OrderID.
ShipmentLog	LogID	TrackingNumber.
AuditLog	AuditLogId	ActorUsername, EntityType, EntityId.
Vehicle	VehicleID	AllowedDrivers, MaintenanceHistory, VehicleEngine.
Warehouse	WarehouseID	Manager.
WarehouseLocation	LocationID	WarehouseID.
WarehouseInventory	LogID	WarehouseID, PackageID, TrackingNumber.
Equipment	EquipmentID	WarehouseID.
VehicleAssignment	AssignmentID	DriverID, VehicleID.

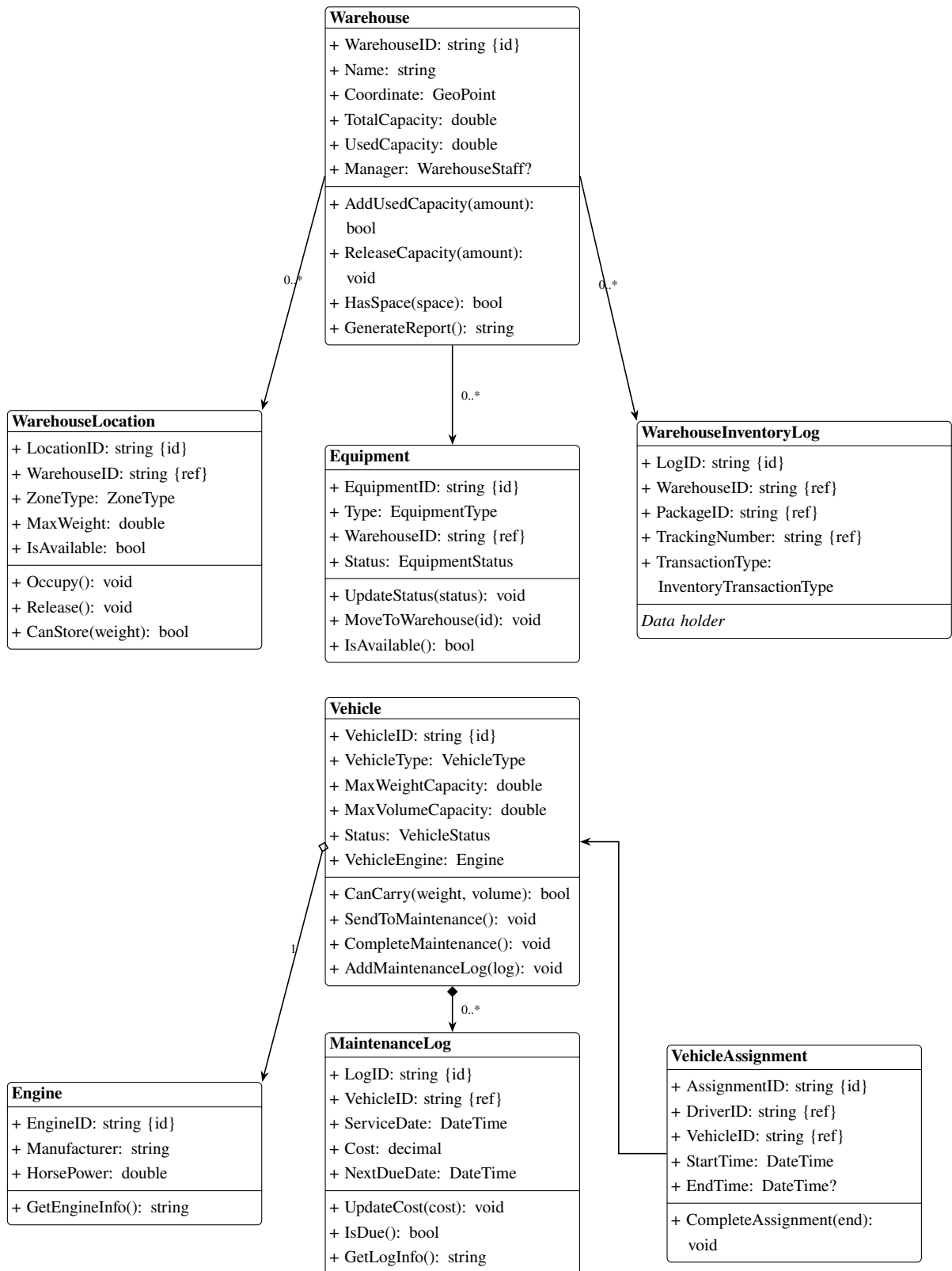
Bảng 2: Các trường định danh và tham chiếu chính trong domain model



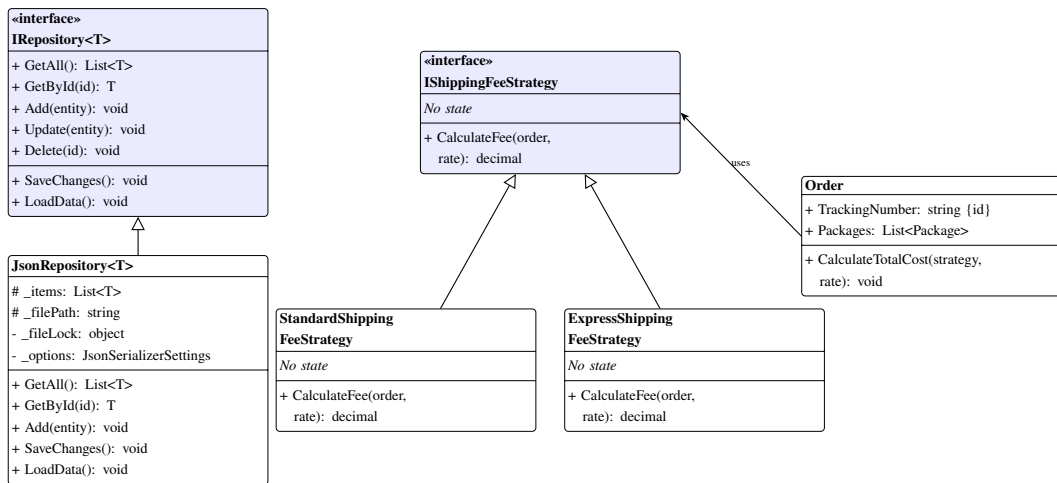
Hình 2: Sơ đồ lớp chi tiết nhóm tác nhân, tài khoản và kiểu giá trị dùng chung



Hình 3: Sơ đồ lớp chi tiết nhóm đơn hàng và nghiệp vụ vận chuyển



Hình 4: Sơ đồ lớp chi tiết nhóm kho, phương tiện và hạ tầng logistics



Hình 5: Sơ đồ lớp chi tiết cho Repository Pattern và Strategy Pattern

2.2.3 Các tính chất cơ bản của Lập trình hướng đối tượng (OOP)

Dự án Logistics được thiết kế dựa trên các nguyên lý lập trình hướng đối tượng nền tảng, giúp hệ thống đạt tính bền vững, dễ bảo trì và khả năng mở rộng tốt. Dưới đây là phân tích chi tiết cách áp dụng bốn trụ cột OOP cùng các ví dụ cụ thể từ mã nguồn.

a. Tính đóng gói (Encapsulation)

Tính đóng gói được thể hiện qua việc che giấu trạng thái nội tại của đối tượng và chỉ cho phép thay đổi dữ liệu thông qua các phương thức công khai (API nghiệp vụ). Các thuộc tính quan trọng trong các lớp thực thể (như Order, Package, Vehicle) đều được khai báo với setter có phạm vi truy cập hạn chế (private set hoặc protected set). Điều này ngăn chặn việc gán giá trị tùy tiện từ bên ngoài lớp, bảo vệ tính toàn vẹn dữ liệu.

Ví dụ, trạng thái của đơn hàng (OrderStatus) và lịch sử thay đổi trạng thái (StatusHistories) không thể chỉnh sửa trực tiếp. Muốn cập nhật trạng thái đơn hàng, đối tượng bên ngoài phải gọi phương thức ChangeStatus(). Phương thức này chịu trách nhiệm:

1. Kiểm tra tính hợp lệ của luồng chuyển đổi trạng thái thông qua hàm kiểm thử tính CanTransition().
2. Ghi nhận lịch sử thay đổi trạng thái vào danh sách tự quản lý (quan hệ Composition).
3. Đồng bộ hóa trạng thái của tất cả các gói hàng thuộc đơn hàng đó (quan hệ Aggregation).
4. Kích hoạt sự kiện OnStatusChanged để thông báo cho các hệ thống quan sát khác (Observer Pattern).

```

public void ChangeStatus(OrderStatus newStatus, string description = "",
    ↪ string changedBy = "System")
{
    // Kiểm tra ràng buộc chuyển đổi trạng thái trước khi thay đổi
    if (!CanTransitionTo(newStatus))
    {
        throw new InvalidOrderException($"Không thể chuyển trạng thái từ
            ↪ {CurrentStatus} sang {newStatus}");
    }

    if (CurrentStatus != newStatus)
    {
        OrderStatus oldStatus = CurrentStatus;

        // Tạo lịch sử thay đổi trạng thái (Composition)
        OrderStatusHistory historyItem = new OrderStatusHistory
        {
            PreviousStatus = oldStatus,
            NewStatus = newStatus,
            ChangedAt = DateTime.Now,
            Location = this.Route != null ? this.Route.ToString() : "Unknown",
            Description = description,
            ChangedBy = changedBy
        };
        StatusHistories.Add(historyItem);

        // Thực hiện cập nhật trạng thái
        CurrentStatus = newStatus;

        // Đồng bộ trạng thái các kiện hàng con
        SynchronizePackageStatuses(oldStatus, newStatus);

        // Phát sự kiện thông báo (Observer)
        OnStatusChanged?.Invoke(this, oldStatus, newStatus);
    }
}

```

Listing 1: Phương thức ChangeStatus thể hiện tính đóng gói trong lớp Order

Tương tự, lớp Package đóng gói trạng thái vị trí kho của nó. Thuộc tính CurrentWarehouseID và CurrentShelfLocation chỉ được cập nhật qua phương thức nghiệp vụ CheckInWarehouse(), bảo đảm rằng khi kiện hàng nằm trong kho thì thông tin vị trí kệ kệ cũng được cập nhật đồng thời một cách an toàn.

b. Tính trừu tượng (Abstraction)

Tính trừu tượng giúp đơn giản hóa thiết kế hệ thống bằng cách tập trung vào những hành vi cốt lõi và bỏ qua chi tiết cài đặt phức tạp. Trong dự án, tính chất này được thể hiện rõ nét qua việc sử dụng các abstract class (Person, Staff) và một hệ thống các interface nghiệp vụ dùng chung.

Hệ thống định nghĩa 3 interface cốt lõi của domain models:

- ITrackable: Định nghĩa giao diện cho các đối tượng có thể theo dõi trạng thái vận hành và tọa độ địa lý. Giao diện này được hiện thực bởi Order, Vehicle, Warehouse, Driver. Người dùng chỉ cần làm việc với kiểu ITrackable để lấy thông tin hành trình mà không cần quan tâm thực thể đó là một chiếc xe tải hay một đơn hàng.
- IReportable: Chuẩn hóa chức năng xuất báo cáo định dạng văn bản cho các thực thể như Order, Warehouse, ProblemReport.
- ISalaryCalculable: Định nghĩa giao diện tính toán lương và giải trình các khoản thu nhập của nhân sự. Giao diện này do lớp abstract Staff kế thừa trực tiếp.

```
public interface ITrackable
{
    string GetCurrentStatus();
    string GetTrackingInfo();
}

public interface IReportable
{
    string GenerateReport();
}

public interface ISalaryCalculable
{
    decimal CalculateSalary();
    string GetSalaryBreakdown();
}
```

Listing 2: Định nghĩa các interface cốt lõi thể hiện tính trừu tượng

Ngoài ra, tính trừu tượng còn được ứng dụng ở tầng truy cập dữ liệu thông qua interface generic IRepository<T>. Tầng dịch vụ (Services) chỉ tương tác với interface IRepository<T> mà không cần biết dữ liệu được lưu trữ thực tế dưới dạng file JSON thông qua lớp triển khai cụ thể JsonRepository<T> hay lưu trong cơ sở dữ liệu quan hệ. Điều này tuân thủ nguyên lý Dependency Inversion trong nguyên tắc thiết kế SOLID.

c. Tính kế thừa (Inheritance)

Kế thừa giúp chia sẻ mã nguồn chung và tạo lập mối quan hệ phân cấp tự nhiên giữa các thực thể trong thế giới thực. Dự án mô hình hóa nhóm đối tượng con người thông qua một cây phân cấp kế thừa nhiều tầng rõ ràng:

- Person (Lớp cha trùu tượng): Định nghĩa các thuộc tính cơ bản của một con người như Id, FullName, PhoneNumber, Email, BirthDay, HomeAddress.
- Customer (Kế thừa từ Person): Bổ sung các thông tin dành riêng cho khách hàng như loại khách hàng (CustomerType), điểm tích lũy (LoyaltyPoints), tọa độ nhận hàng mặc định.
- Staff (Lớp cha trùu tượng kế thừa từ Person và hiện thực ISalaryCalculable): Bổ sung các thông tin quản lý nhân sự như mã nhân viên (StaffID), chức vụ (Role), lương cơ bản (BaseSalary), ngày vào làm.
- Các lớp nhân viên cụ thể (Admin, Driver, Dispatcher, WarehouseStaff) kế thừa trực tiếp từ lớp Staff.

```

public abstract class Person : ISerializable
{
    public string Id { get; set; } = string.Empty;
    public string FullName { get; set; } = string.Empty;
    public string PhoneNumber { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
}

public abstract class Staff : Person, ISalaryCalculable
{
    public string StaffID { get; set; } = string.Empty;
    public Role Role { get; set; }
    public decimal BaseSalary { get; set; }

    // Yêu cầu các lớp con bắt buộc phải triển khai công thức tính lương cụ
    // → thể
    public abstract decimal CalculateSalary();
    public abstract string GetSalaryBreakdown();
}

public class Driver : Staff, ITrackable
{
    public string LicenseNumber { get; set; } = string.Empty;
    public DriverStatus DriverStatus { get; set; }
    public int DeliveryCount { get; set; }
}

```

Listing 3: Khai báo cây phân cấp kế thừa nhóm nhân sự

Nhờ cấu trúc kế thừa này, các thuộc tính chung của con người và nhân sự không bị khai báo lặp lại ở từng lớp con. Khi hệ thống cần bổ sung một vai trò nhân viên mới (ví dụ: nhân viên chăm sóc khách hàng - CustomerServiceStaff), lập trình viên chỉ cần tạo lớp mới kế thừa từ Staff và cài đặt công thức tính lương tương ứng mà không làm xáo trộn mã nguồn sẵn có.

d. Tính đa hình (Polymorphism)

Tính đa hình cho phép các đối tượng thuộc các lớp khác nhau phản hồi cùng một lời gọi phương thức theo những cách thức đặc thù riêng biệt. Trong dự án, tính đa hình được áp dụng tại hai vị trí cốt lõi:

1. Đa hình trong việc tính lương nhân viên (Method Overriding): Lớp chi tiết cụ thể như Driver hoặc Dispatcher sẽ ghi đè (override) phương thức trừu tượng CalculateSalary() của lớp cha Staff để cài đặt công thức tính lương đặc thù của mình. Ví dụ, lớp Driver tính

lương dựa trên lương cơ bản cộng với tiền thưởng theo số chuyến giao hàng thực tế và phụ cấp xăng xe:

```
public override decimal CalculateSalary()
{
    decimal deliveryBonus = DeliveryCount * BonusPerDelivery;
    return BaseSalary + deliveryBonus + FuelAllowance;
}
```

Listing 4: Ghi đè phương thức tính lương của tài xế (Driver)

Khi duyệt qua một danh sách các nhân viên kiểu `List<Staff>`, lời gọi hàm `CalculateSalary()` sẽ kích hoạt công thức tính toán tương ứng của từng đối tượng con tại thời điểm thực thi (Runtime Polymorphism).

2. Đa hình trong việc áp dụng chiến lược tính phí vận chuyển (Interface Polymorphism): Đơn hàng (Order) chứa thuộc tính kiểu giao diện `IShippingFeeStrategy`. Khi thực hiện tính cước phí bằng phương thức `CalculateTotalCost()`, cước phí sẽ được tính toán đa hình tùy thuộc vào chiến lược phí được truyền vào là `StandardShippingFeeStrategy` hay `ExpressShippingFeeStrategy`.

```
public void CalculateTotalCost(IShippingFeeStrategy feeStrategy, decimal
    ↪ baseRatePerKg)
{
    if (feeStrategy != null)
    {
        // Lời gọi đa hình tùy thuộc vào đối tượng cụ thể của chiến lược phí
        TotalCost = feeStrategy.CalculateFee(this, baseRatePerKg);
    }
}
```

Listing 5: Đa hình trong tính phí vận chuyển của đơn hàng

2.2.4 Design Pattern

Hệ thống Logistics áp dụng các mẫu thiết kế hướng đối tượng (GoF Design Patterns) kinh điển để giải quyết các vấn đề thiết kế thực tế, tăng độ linh hoạt và giảm thiểu sự phụ thuộc chéo (coupling) giữa các phân hệ:

a. Strategy Pattern

Bài toán thực tế: Hệ thống cần tính toán cước phí vận chuyển cho đơn hàng dựa trên nhiều loại dịch vụ (Standard, Express, Instant) với các công thức tính toán khác nhau. Nếu lập trình theo cách thông thường, lớp Order sẽ chứa nhiều câu lệnh điều kiện (if-else hoặc

switch-case) phức tạp để rẽ nhánh. Khi cần thêm một loại hình dịch vụ mới, ta bắt buộc phải sửa đổi mã nguồn của lớp Order, điều này vi phạm nguyên tắc Open-Closed (mở rộng tính năng nhưng đóng với việc sửa đổi mã nguồn).

Giải pháp: Áp dụng Strategy Pattern. Hệ thống tách biệt thuật toán tính toán cước phí ra khỏi thực thể đơn hàng bằng cách định nghĩa giao diện IShippingFeeStrategy và các lớp chiến lược cụ thể:

- StandardShippingFeeStrategy: Tính phí đơn giản dựa trên tổng khối lượng tính cước của kiện hàng nhân với đơn giá cơ bản.
- ExpressShippingFeeStrategy: Tính phí có hệ số ưu tiên cao hơn và cộng thêm phụ phí đối với các kiện hàng có khối lượng vượt ngưỡng quy định (hàng siêu trường siêu trọng).

```
IShippingFeeStrategy strategy;  
if (order.ServiceType == ServiceType.Express || order.ServiceType ==  
    ↳ ServiceType.Instant)  
{  
    strategy = new ExpressShippingFeeStrategy(businessRules);  
}  
else  
{  
    strategy = new StandardShippingFeeStrategy();  
}  
order.CalculateTotalCost(strategy, ratePerKg);
```

Listing 6: Ví dụ Strategy tính phí vận chuyển trong OrderService

Khi tính toán cước phí đơn hàng, lớp OrderService sẽ khởi tạo chiến lược tương ứng và truyền vào phương thức CalculateTotalCost() của đơn hàng. Nhờ vậy, khi có thêm chiến lược cước phí mới (ví dụ: giao hàng đông lạnh), ta chỉ cần viết thêm một lớp mới thực hiện giao diện IShippingFeeStrategy mà không phải thay đổi bất kỳ dòng mã nào trong lớp Order.

b. Repository Pattern

Bài toán thực tế: Các dịch vụ xử lý nghiệp vụ (Services) và giao diện WinForms cần thực hiện các thao tác CRUD (Thêm, Đọc, Sửa, Xóa) trên dữ liệu. Nếu các lớp này tự thao tác trực tiếp với các tệp JSON hoặc cơ sở dữ liệu, mã nguồn sẽ bị lặp lại nhiều lần, khó kiểm thử độc lập (unit test) và phụ thuộc chặt chẽ vào cơ chế lưu trữ.

Giải pháp: Áp dụng Repository Pattern để cô lập và chuẩn hóa tầng truy xuất dữ liệu. Interface IRepository<T> định nghĩa các phương thức chung cho mọi thực thể nghiệp vụ. Lớp generic JsonRepository<T> hiện thực interface này để thực hiện việc lưu trữ trên file JSON. Các repository chuyên biệt như OrderRepository, VehicleRepository kế thừa từ

JsonRepository<T> và bổ sung các truy vấn đặc thù (ví dụ: tìm kiếm đơn hàng theo mã khách hàng, tìm xe trống lịch).

Để đảm bảo an toàn dữ liệu và tránh xung đột khi có nhiều tiến trình ghi tệp đồng thời (Thread-Safety), lớp JsonRepository<T> cài đặt cơ chế khóa ghi và lưu tệp an toàn:

1. Sử dụng đối tượng khóa tĩnh lock để đảm bảo tại một thời điểm chỉ có duy nhất một luồng được quyền thực thi phương thức lưu dữ liệu.
2. Khi ghi dữ liệu xuống đĩa cứng, dữ liệu được ghi vào một tệp tạm (.tmp).
3. Sao lưu tệp hiện hành thành tệp sao lưu (.bak).
4. Thay thế tệp hiện hành bằng tệp tạm. Việc này đảm bảo nếu quá trình ghi tệp bị ngắt đột ngột (ví dụ mất điện), dữ liệu cũ vẫn được bảo toàn trong tệp .bak và hệ thống sẽ tự động khôi phục lại khi khởi động lại ứng dụng.

c. Factory Method Pattern

Bài toán thực tế: Việc khởi tạo thủ công các đối tượng repository yêu cầu phải truyền đúng đường dẫn thư mục lưu trữ và tên tệp JSON cấu hình tương ứng. Điều này làm rườm rà logic khởi tạo đối tượng và tạo sự phụ thuộc trực tiếp vào lớp cài đặt cụ thể JsonRepository<T>.

Giải pháp: Áp dụng Factory Method thông qua lớp RepositoryFactory. Phương thức tĩnh CreateRepository<T>() đóng vai trò làm nhà máy sản xuất các đối tượng repository. Giao diện người dùng hoặc các dịch vụ nghiệp vụ chỉ cần gọi Factory này để nhận về giao diện IRepository<T> mong muốn mà không cần biết cách thức tệp lưu trữ được cấu hình bên dưới.

```
public static IRepository<T> CreateRepository<T>(
    string folderPath,
    string fileNameWithoutExt) where T : class
{
    string fullPath = Path.Combine(folderPath, fileNameWithoutExt + ".json");
    // Trả về lớp triển khai cụ thể thông qua kiểu giao diện trừu tượng
    return new JsonRepository<T>(fullPath);
}
```

Listing 7: RepositoryFactory hiện thực Factory Method

d. Observer Pattern

Bài toán thực tế: Khi trạng thái của một đơn hàng thay đổi (ví dụ chuyển từ ReadyForDispatch sang InTransit), hệ thống cần thực hiện nhiều hành động đi kèm như: gửi thông báo cho khách hàng qua email, ghi nhật ký tồn kho tại kho xuất hàng, cập nhật bảng điều khiển (Dashboard) hiển thị tại giao diện điều phối. Nếu gán trực tiếp các xử lý này vào phương thức

ChangeStatus() của lớp Order, lớp này sẽ gánh vác quá nhiều trách nhiệm và khó thay đổi các hành động đi kèm.

Giải pháp: Áp dụng Observer Pattern. Lớp Order đóng vai trò là Subject (đối tượng được quan sát), khai báo một sự kiện (event) C# dựa trên delegate OrderStatusChangedEventHandler:

```
public event OrderStatusChangedEventHandler? OnStatusChanged;
```

Các thành phần khác trong hệ thống như NotificationService, DashboardForm đóng vai trò là các Observer (người quan sát). Chúng sẽ đăng ký lắng nghe sự kiện thay đổi trạng thái của đơn hàng. Khi trạng thái đơn hàng thay đổi, sự kiện OnStatusChanged sẽ tự động kích hoạt (notify) tất cả các phương thức đã đăng ký mà lớp Order không cần biết chi tiết các phương thức đó làm gì. Điều này giúp giảm độ liên kết chặt chẽ giữa các thành phần nghiệp vụ.

e. Singleton/Session Pattern

Để quản lý thông tin phiên đăng nhập của người dùng hiện hành trong suốt vòng đời hoạt động của ứng dụng WinForms, lớp SessionManager được thiết kế theo dạng quản lý trạng thái tập trung duy nhất (Singleton-like). Lớp này cung cấp các thuộc tính tĩnh (static) như CurrentUser và IsLoggedIn để bất kỳ Form giao diện nào cũng có thể nhanh chóng kiểm tra thông tin và phân quyền thực thi chức năng thông qua lớp bảo mật RoleGuard.

f. DTO và Mapping Extension

DTO được dùng để tách dữ liệu hiển thị khỏi các thực thể nghiệp vụ cốt lõi (Models). Các thực thể domain như Order, Customer, Vehicle thường mang nhiều thuộc tính và hành vi phức tạp không cần thiết cho việc hiển thị trực quan lên màn hình. Hơn nữa, việc chuyển giao trực tiếp thực thể nghiệp vụ lên tầng giao diện có thể vô tình làm lộ các chi tiết nghiệp vụ nhạy cảm hoặc tạo liên kết quá chặt chẽ (tight coupling).

Thông qua các DTO (ví dụ: OrderDTO, CustomerDTO) và các lớp mở rộng chuyển đổi (Mappings), dữ liệu được chuẩn bị tinh gọn, tối ưu và giao diện chỉ tương tác với DTO. Điều này giúp bảo vệ tính đóng gói và giữ cho tầng giao diện hoàn toàn độc lập với các quy tắc nghiệp vụ nội bộ.

2.2.5 Các quan hệ trong Lập trình hướng đối tượng

a. Inheritance (Kế thừa)

Quan hệ kế thừa giúp mô hình hóa phân cấp tác nhân:

- Customer kế thừa Person.
- Staff kế thừa Person.
- Admin, Driver, Dispatcher, WarehouseStaff kế thừa Staff.

Trong sơ đồ lớp có 6 quan hệ kế thừa chính. Các quan hệ này giúp gom thuộc tính chung và cho phép các vai trò cụ thể mở rộng hành vi riêng.

b. Realization (Hiện thực interface)

Các lớp hiện thực interface để cam kết có hành vi chung:

- Staff hiện thực ISalaryCalculable.
- Driver, Order, Vehicle, Warehouse, WarehouseLocation, Equipment hiện thực ITrackable.
- Order, Warehouse, ProblemReport hiện thực IReportable.
- JsonRepository<T> hiện thực IRepository<T>.
- Các lớp strategy hiện thực IShippingFeeStrategy.

c. Association (Liên kết)

Association là quan hệ một lớp biết hoặc tham chiếu đến lớp khác nhưng không sở hữu vòng đời của lớp đó. Ví dụ:

- Order liên kết với Customer qua SenderID và ReceiverID.
- Order liên kết với Driver và Vehicle qua AssignedDriverID và AssignedVehicleID.
- Warehouse liên kết với WarehouseStaff qua thuộc tính Manager.
- Transaction, ProblemReport, ShipmentLog liên kết với đơn hàng qua OrderID hoặc TrackingNumber.

d. Aggregation (Tập hợp)

Aggregation là quan hệ “có” nhưng vòng đời đối tượng con có thể độc lập với đối tượng cha. Trong hệ thống:

- Order chứa danh sách Package; kiện hàng thuộc đơn về mặt nghiệp vụ nhưng vẫn được truyền vào thông qua AddPackage().
- Vehicle có Engine; động cơ có thể được tạo độc lập rồi gán vào xe bằng InstallEngine().
- Driver có danh sách AssignedVehicles; tài xế và xe vẫn là hai thực thể độc lập.

e. Composition (Hợp thành)

Composition là quan hệ “có” với vòng đời phụ thuộc chặt vào lớp cha. Trong bài:

- Order tạo DeliveryRoute bên trong constructor, vì tuyến giao hàng này gắn trực tiếp với đơn.

- Order tạo OrderDetail bên trong phương thức AddDetail(), chi tiết đơn không có ý nghĩa nếu tách khỏi đơn.
- Order quản lý OrderStatusHistory; lịch sử trạng thái chỉ tồn tại để mô tả vòng đời của đơn hàng.
- Vehicle quản lý MaintenanceHistory; lịch sử bảo trì được lưu như một phần hồ sơ phương tiện.

f. Dependency (Phụ thuộc)

Dependency là quan hệ sử dụng tạm thời hoặc phụ thuộc chức năng. Ví dụ:

- Order.CalculateTotalCost() phụ thuộc IShippingFeeStrategy.
- OrderService, DispatchService, WarehouseService phụ thuộc repository để đọc/ghi dữ liệu.
- RoleGuard phụ thuộc SessionManager để kiểm tra quyền.
- Validator phụ thuộc model để kiểm tra dữ liệu đầu vào.

2.2.6 Kỹ thuật Serialization và Deserialization

Hệ thống lưu dữ liệu bằng JSON thông qua thư viện Newtonsoft.Json. Các model quan trọng như Person, Order, Vehicle, Warehouse cài đặt ISerializable hoặc dùng JsonProperty để kiểm soát quá trình serialize/deserialize. Constructor không tham số được bổ sung cho JSON, còn constructor nhận SerializationInfo và StreamingContext được dùng để khôi phục dữ liệu khi cần.

```

protected Person(SerializationInfo info, StreamingContext context)
{
    Id = info.GetString("Id") ?? string.Empty;
    FullName = info.GetString("FullName") ?? string.Empty;
    PhoneNumber = info.GetString("PhoneNumber") ?? string.Empty;
    Email = info.GetString("Email") ?? string.Empty;
}

public virtual void GetObjectData(
    SerializationInfo info,
    StreamingContext context)
{
    info.AddValue("Id", Id);
    info.AddValue("FullName", FullName);
    info.AddValue("PhoneNumber", PhoneNumber);
    info.AddValue("Email", Email);
}

```

Listing 8: Ví dụ ISerializable trong Person

Lớp `JsonRepository<T>` dùng generic có ràng buộc `where T : class`, cho phép tái sử dụng cùng một cơ chế lưu trữ cho nhiều loại dữ liệu như `Order`, `Vehicle`, `Customer`, `Warehouse`. Repository cấu hình `TypeNameHandling.Auto` và `LogisticsSerializationBinder` để hỗ trợ dữ liệu có tính kế thừa nhưng vẫn giới hạn kiểu được phép deserialize, giảm rủi ro bảo mật.

Khi lưu dữ liệu, `JsonRepository<T>` dùng cơ chế lock và safe write:

- Khóa file bằng `_fileLock` để tránh đọc/ghi đồng thời gây lỗi dữ liệu.
- Ghi dữ liệu ra file tạm `.tmp`.
- Sao lưu file cũ sang `.bak`.
- Sao chép file tạm thành file chính.
- Nếu file chính lỗi khi đọc, thử khôi phục từ `.bak`; nếu vẫn lỗi thì bảo toàn file lỗi bằng bản `.corrupt`.

Nhờ kỹ thuật này, hệ thống vừa đáp ứng yêu cầu lưu trữ đơn giản của đồ án OOP, vừa có cơ chế an toàn dữ liệu tốt hơn so với việc đọc/ghi JSON trực tiếp trong từng form.

3 XÂY DỰNG ỨNG DỤNG

3.1 Thiết kế giao diện chương trình

3.1.1 Giao diện đăng nhập, đăng ký

3.1.2 Giao diện tổng quan

3.1.3 Giao diện quản lý đơn hàng

3.1.4 Giao diện quản lý khách hàng

3.1.5 Giao diện quản lý nhân sự

3.1.6 Giao diện quản lý phương tiện

3.1.7 Giao diện quản lý kho bãi

3.1.8 Giao diện điều phối giao hàng

3.1.9 Giao diện quản lý chứng từ

3.1.10 Giao diện báo cáo, thống kê

3.2 Phát triển các chức năng của ứng dụng

3.2.1 Quản lý tài khoản và phân quyền

3.2.2 Quản lý khách hàng

3.2.3 Quản lý nhân sự

3.2.4 Quản lý phương tiện

3.2.5 Quản lý kho bãi

3.2.6 Quản lý đơn hàng và kiện hàng

3.2.7 Điều phối tài xế và phương tiện

3.2.8 Quản lý thanh toán và hóa đơn

3.3 Các kịch bản thực thi ứng dụng

4 THẢO LUẬN & ĐÁNH GIÁ

4.1 Các kết quả nhận được

4.2 Một số tồn tại

4.3 Hướng phát triển

Tài liệu tham khảo

- [1] Clark, D. (2015). *Beginning C# object-oriented programming* (2nd ed.). Apress.
- [2] Harwani, B. M. (2015). *Learning Object-Oriented Programming in C# 5.0*. Nelson Education.
- [3] Bishop, J. (2007). *C# 3.0 Design Patterns: Use the Power of C# 3.0 to Solve Real-World Problems*. O'Reilly.
- [4] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [5] Lehman, M. M., & Belady, L. A. (1985). *Program Evolution: Processes of Software Change*. Academic Press.
- [6] Microsoft Corporation. (2024). *ASP.NET Core documentation*. Available: <https://docs.microsoft.com/en-us/aspnet/core>
- [7] Meta Platforms. (2024). *React – A JavaScript library for building user interfaces*. Available: <https://reactjs.org>
- [8] Microsoft Corporation. (2024). *Entity Framework Core documentation*. Available: <https://docs.microsoft.com/en-us/ef/core>
- [9] Khoa Công nghệ Thông tin, Trường Đại học Kinh tế - Tài chính TP.HCM. (2026). *Slide bài giảng môn Lập Trình Hướng Đối Tượng*.

Phụ lục

Phụ lục hình ảnh

Bảng phân công nhiệm vụ

Bảng phân công công việc của nhóm khá chi tiết nên được nhóm lưu trữ trực tuyến để thuận tiện theo dõi và cập nhật:

XEM BẢNG PHÂN CÔNG CÔNG VIỆC

Link đề án

Mã nguồn của hệ thống được lưu trữ trên GitHub để thuận tiện cho việc theo dõi, phát triển:

 **XEM SOURCE CODE HỆ THỐNG**